38. Design Distributed Cache

Key Terms:

Term	Full Name	What it does
LRU	Least Recently Used	Eviction policy removing oldest accessed item
LFU	Least Frequently Used	Eviction policy removing least accessed item
TTL	Time To Live	Auto-expiration after set duration
Consistent Hashing	Distribution algorithm	Minimizes data movement when scaling
Virtual Node	Hash ring technique	Multiple ring positions per physical node
QPS	Queries Per Second	Throughput measure for cache
AOF	Append-Only File	Redis persistence by logging writes
RDB	Redis Database	Redis point-in-time snapshot persistence

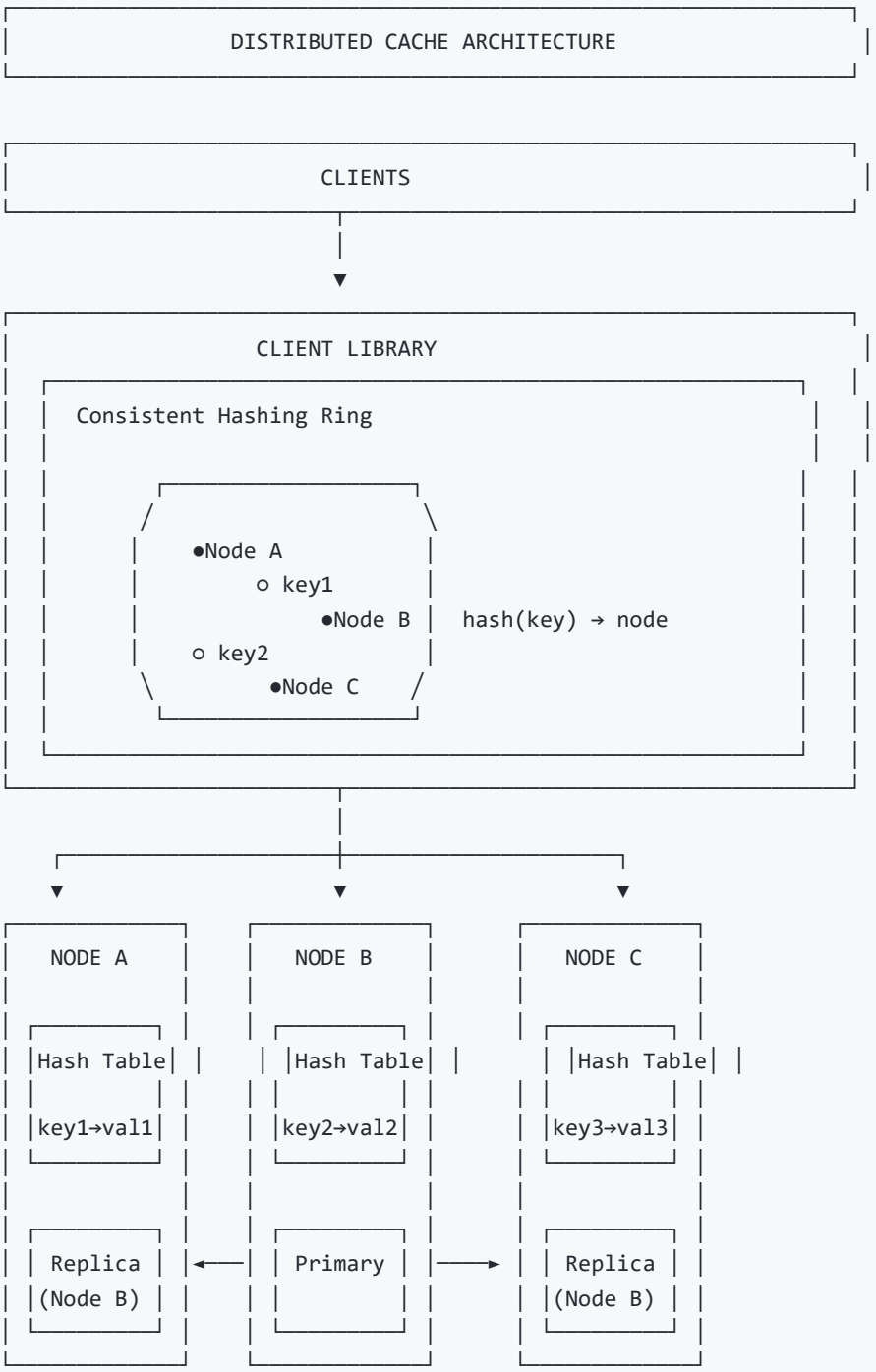
Problem Statement

Design a distributed caching system like Redis or Memcached that provides low-latency data access across multiple nodes.

Clarifying Questions to Ask

1. **Scale**: Data size? QPS? (1TB, 1M QPS?)
2. **Consistency**: Strong or eventual?
3. **Eviction**: LRU, LFU, TTL?
4. **Persistence**: In-memory only or persist to disk?
5. **Data types**: Key-value only or complex types (lists, sets)?
6. **Replication**: Primary-replica or peer-to-peer?
7. **Partition tolerance**: How to handle network splits?

High-Level Design



API Design

Cache Operations APIs

Method	Endpoint	Description
`GET`	`/api/v1/cache/{key}`	Get cached value
`PUT`	`/api/v1/cache/{key}`	Set cache entry with TTL

`DELETE`	`/api/v1/cache/{key}`	Delete cache entry
`POST`	`/api/v1/cache/batch/get`	Batch get multiple keys
`POST`	`/api/v1/cache/batch/set`	Batch set multiple keys

Cache Set Request

```
PUT /api/v1/cache/user:123:profile
{
  "value": { "name": "John", "email": "john@example.com" },
  "ttl_seconds": 3600,
  "policy": "write-through"   // write-through, write-behind, write-around
}
```

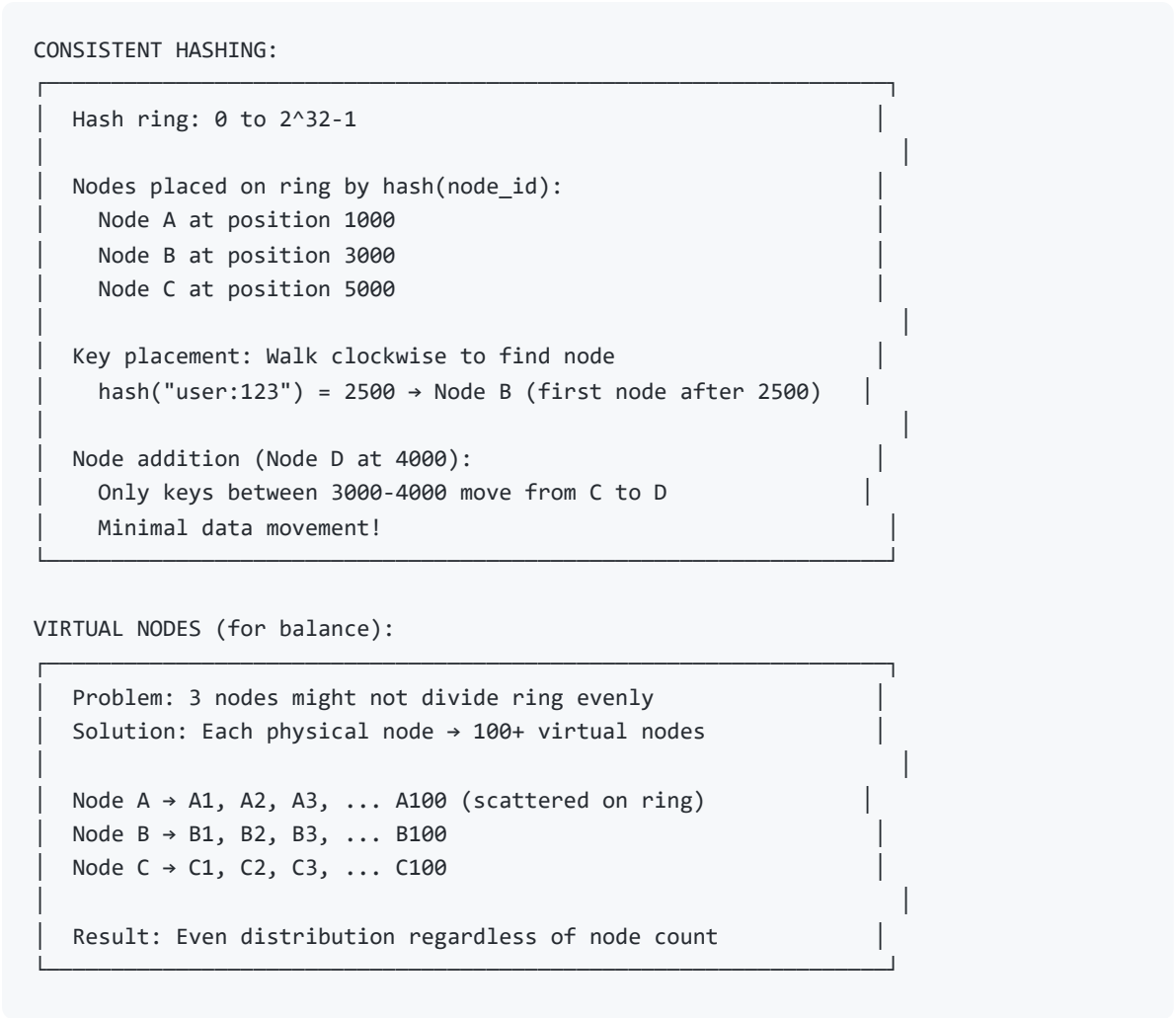
Cache Management APIs

Method	Endpoint	Description
`POST`	`/api/v1/cache/invalidate`	Invalidate by pattern (e.g., `user:123:*`)
`GET`	`/api/v1/cache/stats`	Get hit rate, memory usage, eviction count
`GET`	`/api/v1/cache/nodes`	List cache nodes and distribution
`POST`	`/api/v1/cache/flush`	Flush entire cache (admin only)

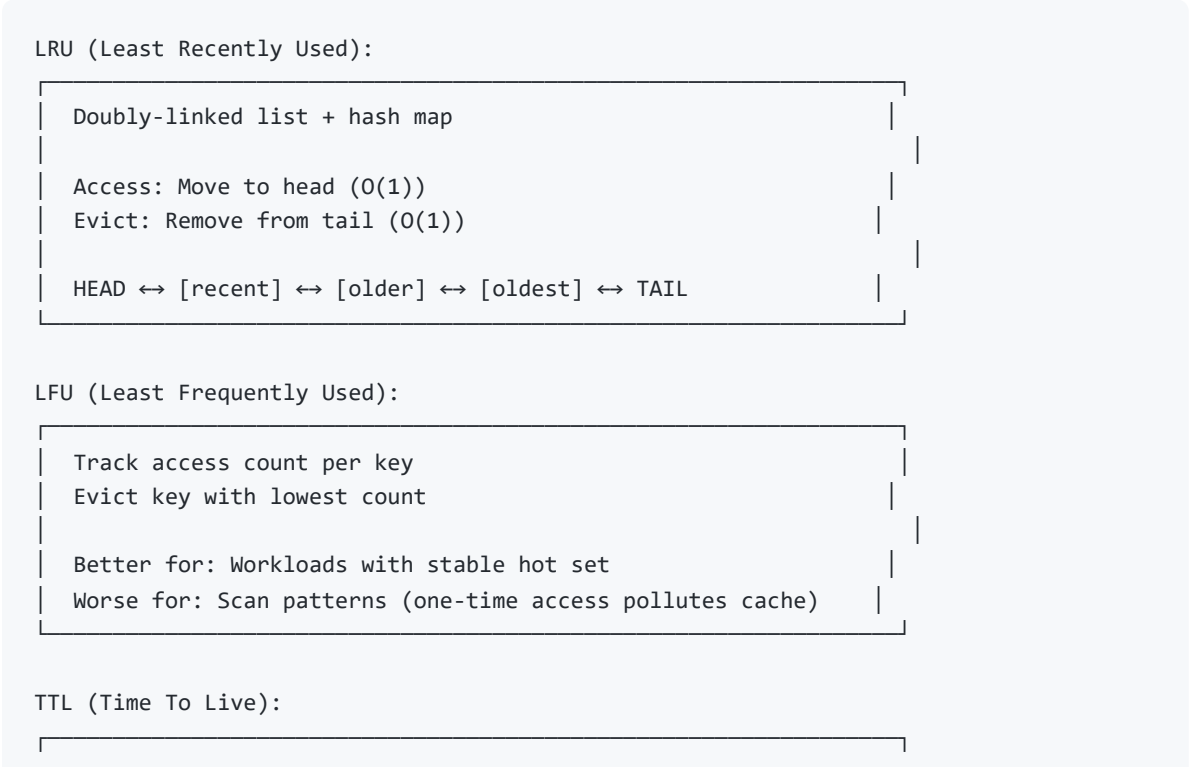
Key Components

Component	Purpose	Technology
Client Library	Routing, connection pooling	Custom SDK
Cache Node	Store data in memory	Custom or Redis
Consistent Hashing	Distribute keys	Client-side
Replication	Data redundancy	Async replication
Cluster Manager	Node membership	Gossip protocol

Consistent Hashing



Eviction Policies



```
│  Each key has expiration timestamp                          │
│  Background thread cleans expired keys                      │
│  Lazy expiration: Check on access                           │
└─────────────────────────────────────────────────────────────┘
```

Scalability Considerations

- **Horizontal scaling**: Add nodes, consistent hashing redistributes keys
- **Hot keys**: Replicate hot keys to multiple nodes
- **Memory limits**: Set max memory, configure eviction policy
- **Connection pooling**: Reuse connections in client library
- **Pipelining**: Batch multiple commands in single round trip

Trade-offs

Decision	Option A	Option B	Recommendation
Consistency	Strong	Eventual	**Eventual** - for cache
Eviction	LRU	LFU	**LRU** - simpler, good enough
Replication	Sync	Async	**Async** - lower latency
Persistence	None	AOF/RDB	**Optional** - depends on use case

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Redis	In-memory cache with persistence options	Tech Guide
Cosmos DB	Session store with global distribution	Tech Guide
Consistent Hashing	Key distribution across cache nodes	Tech Guide

---

How to Explain It Well

> **Opening**: "Distributed cache needs consistent hashing for even distribution and minimal data movement when scaling."

> **Key talking points**:
>
> - "Consistent hashing: add node = move only 1/N of keys"
> - "Virtual nodes ensure even distribution across physical nodes"
> - "LRU eviction with doubly-linked list for O(1) operations"
> - "Async replication to replicas for fault tolerance"
> - "Client library handles routing - no proxy needed"
